Temporizadores

GPIO18 no es el pin físico número 18 de la placa. Los GPIO son nombres de las señales, en el DevKit pueden estar en otra posición.

Recordar: GPIO34 al 39 solamente entrada. Y una variable static dentro de loop guarda el valor entre vueltas.

El timer es un periférico de hardware que cuenta pulsos de reloj. No es un for que esté sumando con la CPU, el circuito cuenta mientras el programa puede hacer otra cosa.

Si el loop usa millis() tiene que ir preguntando la hora. Con un temporizador de hardware se puede configurar una alarma y recibir el aviso.

Tiene registros de control, contador y alarma. La CPU lo inicia pero después los ticks se van sumando en el propio periférico.

El reloj y el divisor

En el ejemplo del ESP32 clásico, el reloj APB es 80 MHz. Son 80 millones de pulsos en un segundo, cada pulso equivale a 12,5 ns.

Prescaler es el divisor. Si N = 80, entonces 80 MHz / 80 = 1 MHz.

A 1 MHz, cada cuenta o tick dura 1 microsegundo.

Resolución = 1 / frecuencia de tick. Si divido más, cada tick dura más tiempo, osea menos resolución.

No confundir frecuencia del CPU con frecuencia del timer, son relojes y divisores que pueden estar configurados diferente.

El número de alarma A es hasta dónde quiero contar.

Tiempo = alarma / frecuencia de tick.

Alarma = tiempo × frecuencia de tick.

Si quiero 0,5 s con timer de 1 MHz, alarma 500000. Son 500000 microsegundos.

2000 ticks con 1 MHz son 2 ms.

10 ms necesitan 10000 ticks.

Con 10 kHz y alarma 2500: 0,25 s, da 4 eventos por segundo.

Otro ejercicio: frecuencia de 40 kHz y alarma 40000 = 1 segundo. Desde 80 MHz el prescaler sería 2000.

Con las unidades hay que fijarse si está en MHz, kHz, ms o µs. Ahí es donde uno se equivoca.

Alarma y desbordamiento

Autorrecarga: al llegar a la alarma vuelve a arrancar de la forma configurada y así se repite cada intervalo.

Desbordamiento es diferente: el contador llegó hasta el número más grande que puede representar. En 8 bits el máximo es 255, en 16 bits 65535.

La alarma la eligió el programa, el overflow viene por el tamaño del contador.

ESP32 clásico con TIMG0 y TIMG1, dos temporizadores por grupo, en total cuatro timers generales de 64 bits.

ATmega328P tiene Timer0 y Timer2 de 8 bits, Timer1 de 16 bits. Cambia la cantidad de bits y los prescalers.

En estos timers el contador puede ser grande, entonces el problema casi nunca es que llegue al límite en ejercicios pequeños.

Usar el timer desde Arduino

Una API son funciones que nos dejan utilizar el hardware sin escribir todos los registros. El core es el paquete que se instala para que Arduino pueda compilar y cargar código de esa placa.

La API del ESP32 cambió de la versión 2.x a 3.x. Por eso hay ejemplos viejos de temporizadores que no compilan con el core nuevo.

Con core 3.x hay que ver bien las funciones para crear timer, poner alarma y registrar interrupción. No copiar mezclando versiones.

Cuando llega a la alarma, el hardware puede generar interrupción y la CPU ejecuta una ISR. Después regresa a lo que estaba haciendo.

ISR = rutina de interrupción. No debería durar mucho porque el programa deja de hacer otras tareas mientras atiende ese aviso.

En vez de imprimir ahí, mejor subir una bandera y que loop haga Serial.println o encienda el LED.

IRAM_ATTR tiene que ver con tener el código disponible en RAM de instrucciones. volatile para una variable que cambia fuera del flujo normal, pero tampoco protege por sí solo varias operaciones a la vez.

Si un temporizador tiene resolución de 1 µs no significa que el código de la ISR se ejecute exactamente al microsegundo, puede haber latencia.

El contador sigue avanzando aunque el loop esté ocupado. La parte que atiende la interrupción depende de las prioridades y del estado del sistema.

Quedó revisar la fórmula y distinguir alarma de overflow.